

PACT: Post-route Agentic Checkpoint Tuning for FPGA Timing Closure

Huan Lin*, Kunlong Li*, Lingli Wang, and Zhiang Wang

College of Integrated Circuits and Micro-Nano Electronics

Fudan University, Shanghai, China

{huanlin24, klli24}@m.fudan.edu.cn, {llwang, zhiangwang}@fudan.edu.cn

Abstract—Late-stage FPGA timing closure often starts from an implemented design whose remaining violations are visible in timing reports. Engineering change order (ECO) optimization is a standard mechanism for applying localized changes to such designs without restarting the full implementation flow. Automating post-route ECO optimization remains challenging. A post-route change must improve timing without violating routing legality, hold or pulse-width timing constraints, checkpoint replayability or functional equivalence. This paper presents PACT, a *Post-route Agentic Checkpoint Tuning* framework for Vivado design checkpoints (DCP). PACT represents post-route tuning as *validation-gated transitions* between accepted and candidate checkpoints. From checkpoint-derived evidence, an agent selects localized backend actions through a profile-driven recipe planner over typed Vivado and RapidWright skills, and probes tool behavior in isolated workspaces. PACT records each trial as an evidence-gated case to guide candidate generation and suppress unsafe, unsupported or ineffective actions. Across 35 UltraScale+ post-route checkpoints, PACT improves *validation-clean* $F_{\max}$ (maximum operating frequency) by a geometric mean of +22.30% over the original implementations, compared with +15.14% for DATuner and +9.78% for the Codex Agent. On shared designs, PACT achieves these gains $6.4\times$ faster than the uncapped DATuner and at an average token cost of only \$0.16 per DCP ($24.5\times$ lower than the free-form Codex Agent). The source code is available in an anonymous repository [37].

I. INTRODUCTION

Field-programmable gate arrays (FPGAs) provide flexible acceleration by mapping application-specific parallelism onto reconfigurable hardware. Their performance depends not only on the source design but also on backend implementation. During implementation, logic optimization, placement, routing and physical optimization together determine the final sign-off frequency. For large or timing-critical designs, engineers reach timing closure through repeated backend iterations: they inspect timing reports, adjust constraints or implementation strategies, and rerun expensive place-and-route steps until the design meets its target frequency [1]–[3]. This process is costly because each full-flow rerun may discard a legal placed-and-routed state that already contains useful timing, placement and routing information. Late-stage and incremental methods therefore aim to preserve this state while repairing the remaining timing violations.

Engineering change order (ECO) is a standard late-stage mechanism for modifying an existing implemented design. An ECO applies a small change after major implementation steps

have finished, rather than rebuilding the design from scratch. Its goal is to fix a remaining functional, timing or physical issue with limited perturbation and renewed validation [4]–[6]. In FPGA flows, this model can be realized through post-route checkpoints, where a Vivado Design Checkpoint (DCP) preserves the implemented netlist, constraints, placement, routing and physical context. With Vivado and RapidWright support, ECO-style post-route checkpoint tuning can inspect, edit, route and validate an implemented FPGA state instead of blindly rerunning the full flow [1], [7], [8].

This ECO-style formulation also exposes a difficult automation problem. Post-route ECO tuning has traditionally relied on expert-driven iteration: engineers read timing and physical evidence, infer the cause of a violation, apply a local fix and validate the modified checkpoint. The available actions are local, fine-grained and highly design-dependent. Typical actions involve localized physical optimization, path-specific repairs, net rerouting, cell replication or other checkpoint-level changes. The modified checkpoint must then preserve routing legality, timing consistency and checkpoint usability: an edit that improves one critical path may degrade another, and a locally valid change may still produce routing conflicts, constraint mismatches or non-replayable behavior. **The central challenge is to automate the time-consuming and expert-driven ECO process without compromising the rigorous validation required for implemented FPGA states.**

Existing FPGA flow-tuning methods automate a different level of the problem. Prior work has explored scripted flows, Bayesian optimization, multi-armed bandits and learning-assisted autotuning for FPGA implementation recipes [9]–[13]. These methods can reduce manual effort when the search space is defined by directives, seeds, strategies or tool parameters. However, they usually treat the backend flow as a black-box mapping from settings to final quality of results. They do not directly decide which local path, net, cell or region should be modified inside an already routed checkpoint. They also do not maintain a validation record for attempted ECO-style actions. This limits their ability to exploit the detailed evidence available in an implemented design.

LLM-based EDA agents offer a promising decision layer for this setting. Recent systems show that large language models can read design reports, retrieve domain knowledge, invoke EDA tools and refine decisions across iterations [14]–[18]. These abilities match the evidence-rich nature of post-route tuning. However, general tool-using agents are not sufficient

*Huan Lin and Kunlong Li contributed equally to this work.

by themselves. Post-route ECO actions must be tied to a specific accepted checkpoint, executed in isolation, validated before promotion, and recorded when they fail. Without this state discipline, an agent may repeat ineffective actions, modify the wrong design state or accept an unsafe checkpoint.

This paper presents PACT, a post-route agentic checkpoint-tuning framework for FPGA. PACT turns ECO-style checkpoint editing into a closed-loop optimization process over implemented FPGA states. At each iteration, an agent reads checkpoint-derived timing and physical evidence, selects a structured backend action, executes it using a typed Vivado or RapidWright skill, and promotes the result only after validation. A profile-driven recipe planner maps timing-profile evidence to localized optimization actions. A constrained backend harness isolates exploration, enforces rollback and records attempted cases within each run. This design combines the flexibility of LLM-based reasoning with the execution discipline required for post-route ECO optimization. Our main contributions are summarized as follows.

- We introduce an agentic ECO-style post-route optimization framework for FPGA timing closure. The framework operates directly on implemented DCPs and converts local backend edits into validation-gated checkpoint transitions. Upon acceptance of the paper, the source code will be open-sourced to support reproducibility and facilitate further research and development.
- We pair a *profile-driven recipe planner* with a typed library of Vivado and RapidWright skills. The planner maps a checkpoint’s timing profile to effective localized backend actions.
- We design a *constrained backend harness* for validation-gated execution. The harness isolates exploration, promotes only validated actions, and suppresses unsafe or repeated trials with a within-run case record.
- We evaluate PACT on 35 UltraScale+ post-route checkpoints. PACT achieves a geometric-mean validation-clean $F_{\max}$ improvement of +22.30%, compared with +15.14% for DATuner and +9.78% for a free-form Codex Agent.

The remainder of this paper is organized as follows. Section II reviews related work and formalizes the post-route checkpoint tuning problem. Section III presents the PACT framework. Section IV describes the implementation and evaluation. Section V concludes the paper.

II. RELATED WORK

This section reviews four areas most relevant to PACT: FPGA backend and checkpoint infrastructure, ECO optimization, LLM-based EDA agents and automated FPGA flow tuning. It then formulates post-route checkpoint tuning as a validation-gated state-transition problem (Section II-E). Table I compares representative work along the dimensions relevant to PACT.

TABLE I
FEATURE COMPARISON OF REPRESENTATIVE FPGA TUNING, BACKEND CAD AND EDA-AGENT WORK.

Work	FPGA	QoR	Search	Tool	Learn.	Agent	Phys.	Impl.	Know.	Safe
InTime [11]	✓	✓	✓	✓	✓	–	–	–	–	–
DATuner [12]	✓	✓	✓	✓	✓	–	–	–	–	–
LAMDA [13]	✓	✓	✓	✓	✓	–	–	–	–	–
RLPlace [19]	✓	✓	✓	–	✓	–	✓	–	–	–
RWRoute [8]	✓	✓	–	–	–	–	✓	–	–	–
AutoChip [20]	–	–	✓	✓	✓	✓	–	–	–	–
VerilogCoder [21]	–	–	✓	✓	✓	✓	–	–	✓	–
TimelyHLS [22]	✓	✓	✓	✓	✓	✓	–	–	✓	–
LAAFD [23]	✓	✓	✓	✓	✓	✓	–	–	–	–
ORFS-agent [16]	–	✓	✓	✓	✓	✓	✓	–	–	–
RSR [17]	–	✓	✓	✓	✓	✓	✓	–	✓	–
TA Agent [18]	–	✓	–	✓	✓	✓	–	–	✓	–
PACT	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Notes. **QoR**: timing/performance/QoR-oriented optimization. **Search**: automated or iterative exploration. **Tool**: uses EDA-tool feedback, reports, simulation or implementation results as optimization signals. **Learn.**: machine learning (ML), reinforcement learning (RL) and LLM-based decision making. **Agent**: agentic LLM workflow. **Phys.**: backend or physical-design-level action. **Impl.**: optimization on an already implemented post-route checkpoint or state. **Know.**: explicit knowledge, retrieval, reflection or memory. **Safe**: validation-gated optimization with rollback.

A. FPGA Backend and Checkpoint Infrastructure

Commercial FPGA implementation flows provide the core backend operations needed for timing closure, including optimization, placement, physical optimization and routing, together with signoff analysis and checkpoint-based implementation support [1]–[3], [9], [10]. RapidWright complements these capabilities by exposing APIs for reading, writing and manipulating Vivado DCPs [7]. It also supports inspection of implemented designs and timing-driven routing [8]. Together, these capabilities enable post-route checkpoint editing without rebuilding the design from RTL. PACT uses this infrastructure to execute backend edits, but focuses on how an optimizer manages the checkpoint-tuning process.

B. Engineering Change Order Optimization

ECO methods address late-stage functional, timing or physical problems through localized modifications to an implemented design. For FPGA checkpoints, such modifications can include localized physical optimization, net rerouting, inserted logic, cell replication and checkpoint repair. Despite their different mechanisms, these methods share the goal of improving signoff quality while minimizing perturbation to the existing implementation.

Recent work has introduced learning-based decision making into ECO optimization. Jiang et al. [4] combine reinforcement learning with gate sizing for IR-aware timing repair. Azadi et al. [5] use graph edits to generate compact ECO patches at the high-level synthesis (HLS) intermediate-representation level. These studies demonstrate the value of targeted, state-preserving changes, but focus on specific abstraction levels or repair mechanisms.

For FPGA designs, checkpoint-based frameworks provide mechanisms for applying ECO-style changes to implemented designs. RapidWright, for example, supports inserting and routing new logic in an existing placed-and-routed design without full reimplementation [6]. However, such frameworks mainly provide APIs for applying edits to an implemented checkpoint. They do not provide an automated strategy for selecting ECO actions or comparing candidate checkpoints. Post-route ECO tuning therefore still relies heavily on expert-driven iteration. Engineers inspect timing evidence, choose a local edit, validate the candidate checkpoint and decide whether to keep or discard it.

C. LLM-Based EDA Agents

Recent LLM-based EDA agents [14], [15], [20]–[23] demonstrate that language models can interpret design reports, retrieve domain knowledge, invoke EDA tools and refine decisions through iterative feedback. Most of these systems focus on RTL generation, functional debugging or HLS optimization, and therefore operate before physical implementation. Backend-oriented agents [16]–[18] extend LLM-based reasoning to physical-design reports, timing analysis and tool-flow orchestration. However, they mainly target ASIC-oriented flows or analysis tasks. None of the reviewed systems directly optimizes post-route FPGA checkpoints.

D. Automated FPGA Flow Tuning

Automated FPGA design-space exploration reduces manual timing-closure effort by searching over implementation strategies, directives, seeds and tool parameters. InTime, DATuner and LAMDA [11]–[13] use learning or search to prioritize expensive implementation trials and improve timing convergence. Other work formulates placement or routing as sequential decision-making problems [19], [24], [25]. These approaches either tune global implementation settings or optimize decisions within a fixed CAD engine. They do not select localized modifications to paths, nets, cells or regions in an already routed FPGA checkpoint based on semantic information extracted from EDA reports.

In summary, prior work provides checkpoint-manipulation mechanisms, ECO-style local repair, automated flow search and LLM-based EDA reasoning. However, these ingredients have not been combined at the post-route FPGA checkpoint level. A suitable optimizer must select local ECO actions from timing evidence, execute them through constrained backend tools, validate the resulting checkpoint and remember failed or unsafe attempts. PACT addresses this gap with an agentic, validation-gated checkpoint-tuning formulation, which we formalize next.

E. Problem Formulation

Given an initial post-route FPGA checkpoint D_0 , post-route backend optimization applies a sequence of local actions to implemented design states. Applying an action a_t to the current checkpoint D_t produces

$$D_{t+1} = a_t(D_t). \quad (1)$$

Unlike flow-level tuning, these actions modify an already placed-and-routed design rather than restarting implementation from RTL or synthesis.

Within a tuning budget B , the objective is to find a reachable checkpoint with the highest operating frequency:

$$\begin{aligned} D^* &= \arg \max_{D \in \mathcal{R}_B(D_0)} F_{\max}(D) \\ \text{s.t. } &\text{Validate}(D, C) = \text{true}, \end{aligned} \quad (2)$$

where $\mathcal{R}_B(D_0)$ is the set of checkpoints reachable within the budget. The validation set C covers routing legality, hold and pulse-width timing and checkpoint usability; functional equivalence is ensured by construction (Section III-C), not checked in C . The main challenge is to select local actions that improve timing without invalidating the implemented design. PACT addresses this problem as described in the next section.

III. PROPOSED METHOD

This section presents PACT, an agentic framework for post-route FPGA ECO. It first gives a system overview (Section III-A), then describes ECO diagnosis and profile-driven planning (Section III-B), the structured ECO action space (Section III-C) and the constrained execution loop (Section III-D), and finally the validation, acceptance and termination rules (Section III-E).

A. System Overview

As Figure 1 shows, a single iteration works through four stages over the implemented design. The diagnosis stage extracts timing, routing and physical evidence from the accepted checkpoint and summarizes it as a timing profile, a structured description of the current timing failure. The agentic planner uses this profile to rank candidate actions from a typed ECO action library (Table II). PACT then selects the top-ranked action and evaluates it through the matching Vivado or RapidWright skill. The validation stage promotes the result only when it is a fully routed checkpoint that improves setup timing; otherwise PACT keeps the accepted checkpoint. Throughout, PACT separates reasoning from execution. The agent selects and parameterizes actions but never edits the checkpoint itself; every backend change passes through the constrained executor under the same validation policy.

B. ECO Diagnosis and Profile-Driven Planning

A tuning run receives an input checkpoint D_0 , a validation policy C and a budget B . PACT regenerates its reports from the current checkpoint so that the evidence always matches the accepted state. Global setup metrics come from `report_timing_summary`, critical-path detail from `report_timing`, routing status from `report_route_status` and fanout from Vivado net queries. RapidWright supplies object-level detail when finer inspection is needed. PACT condenses this evidence into the timing profile, which records worst and total negative slack, critical start and end points, logic and routing delay, high-fanout nets and routing problems. The run state also holds the

Architecture of PACT for Constrained Post-Route FPGA Backend Optimization

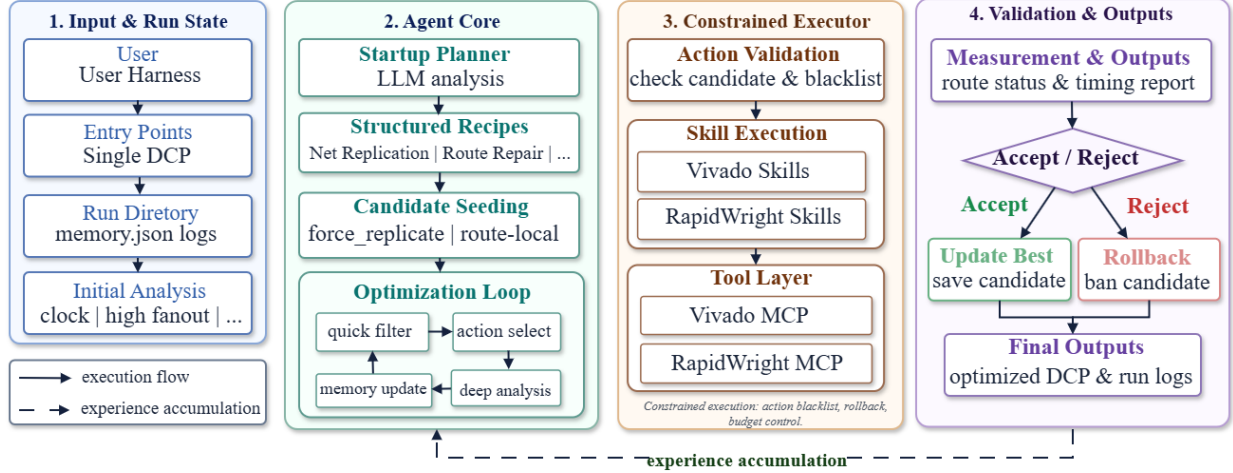


Fig. 1. System overview of PACT: a checkpoint-derived run state feeds an agent core that proposes structured candidate actions, a constrained executor runs only admissible actions through Vivado or RapidWright skills and the validation stage accepts only validation-clean improvements.

accepted checkpoint, its report bundle, the remaining budget, the candidate queue and a within-run evidence record.

The profile-driven planner links timing bottlenecks to executable backend actions. The recipe library P maps common bottlenecks to the action families in Table II. For example, high fanout together with route-dominated delay suggests driver replication, whereas excessive routing delay on a critical net suggests local rerouting. These mappings provide candidate actions rather than a fixed execution schedule.

The agent ranks and parameterizes the applicable candidates from the complete run state. When the diagnosis is incomplete, it may issue non-mutating diagnostics, read-only probes that gather more evidence without changing the accepted checkpoint. The within-run record M suppresses exact repeats and deprioritizes actions that already failed on the same accepted state. Because planning and execution communicate only through this structured interface, the agentic planner is replaceable: the same action library, budget and validation policy run unchanged under either the agent or a deterministic rule-based selector.

C. Structured ECO Action Space

PACT’s action space comprises the nine families in Table II, partitioned into local ECO actions, global fallbacks and non-mutating diagnostics. Local actions repair a specific path, net, cell group or region. Global fallbacks broaden the search when local evidence is weak or local candidates are exhausted. Diagnostics gather evidence but do not themselves improve timing.

A concrete action is a typed instance of one family, fixing its operation type, target objects, parameters, backend, preconditions, required checks and estimated cost. A replication action, for example, names the driver net and sink partition, and a rerouting action the selected net window. The executor can therefore inspect an action before execution and tie the

TABLE II
THE PACT ACTION FAMILIES AND THE EVIDENCE THAT TRIGGERS THEM.
BACKENDS: V = VIVADO AND RW = RAPIDWRIGHT.

Family	Trigger and operation	Tool
<i>Local ECO actions</i>		
Fanout replication	High fanout, route-dominated delay; replicate driver, split sinks, repair routing.	V, RW
Local rerouting	High route delay on critical nets; unroute and reroute a bounded window.	V
Local re-placement	Cell-local path delay; move cells or endpoints, repair affected routes.	V, RW
Logic restructuring	LUT-dominated critical cone; LUT-cone or input-pin transform.	V, RW
Floorplanning	Congestion or long inter-region paths; adjust a pblock, reimplement region.	V, RW
<i>Global fallback actions</i>		
Directive sweep	Weak or diffuse evidence; sweep post-route opt and routing directives.	V
Re-place/route	Local candidates exhausted; timing-driven place-and-route from current DCP.	V
Clock retargeting	Need stronger pressure; tighten temporarily, validate under original clock.	V
<i>Non-mutating diagnostics</i>		
Control & probing	Import, report or inspect objects; never mutates the checkpoint.	V, RW

result to it afterward. The library excludes intentional functional changes by construction: every mutating action is a logic- or timing-equivalent transform, and PACT never counts relaxed timing constraints as an improvement. All reported timing therefore uses the original clock and timing-exception constraints.

D. Constrained Execution and Optimization Loop

Before execution, PACT screens the selected action for admissibility. An action is rejected if its type is unsupported, required targets are missing, preconditions fail, its estimated cost exceeds the remaining budget, or M marks it as duplicate or contraindicated. A proposal that fails these checks never reaches the backend tools. Each supported operation is a

Algorithm 1 Post-route ECO optimization in PACT.

Require: input DCP D_0 , validation policy C , budget B , recipe library P

```
1:  $M \leftarrow \emptyset$ 
2:  $(R_{\text{best}}, v_0) \leftarrow \text{EVALUATE}(D_0, C)$ 
3: if not  $v_0$  then
4:   return input validation failure
5: end if
6:  $D_{\text{best}} \leftarrow D_0$ 
7:  $S \leftarrow \text{BUILDSTATE}(D_{\text{best}}, R_{\text{best}}, M, B)$ 
8:  $Q \leftarrow \text{PLAN}(S, P, M)$ 
9: while  $\text{WITHINBUDGET}(B)$  and  $Q \neq \emptyset$  do
10:   $a \leftarrow \text{POPBEST}(Q)$ 
11:  if not  $\text{ADMISSIBLE}(a, S, C, M)$  then
12:     $M \leftarrow \text{RECORDSKIP}(M, a)$ 
13:    continue
14:  end if
15:   $D' \leftarrow \text{EXECUTEISOLATED}(D_{\text{best}}, a)$ 
16:  if  $D'$  is unavailable then
17:     $M \leftarrow \text{RECORDFAILURE}(M, a)$ 
18:  else
19:     $(R', v') \leftarrow \text{EVALUATE}(D', C)$ 
20:    if  $v'$  and  $\text{SCORE}(R') > \text{SCORE}(R_{\text{best}}) + \epsilon$  then
21:       $D_{\text{best}} \leftarrow D'$ 
22:       $R_{\text{best}} \leftarrow R'$ 
23:       $M \leftarrow \text{RECORDACCEPT}(M, a, R')$ 
24:    else
25:       $\text{DISCARD}(D')$ 
26:       $M \leftarrow \text{RECORDREJECT}(M, a, R')$ 
27:    end if
28:  end if
29:   $S \leftarrow \text{BUILDSTATE}(D_{\text{best}}, R_{\text{best}}, M, B)$ 
30:   $Q \leftarrow \text{UPDATEPLAN}(Q, S, P, M)$ 
31: end while
32: return  $D_{\text{best}}, R_{\text{best}}$  and  $M$ 
```

typed skill that wraps a controlled subset of Vivado Tcl or RapidWright calls. The skill runs on a copy of the accepted checkpoint in an isolated trial directory and returns a fixed artifact set: an output DCP, timing reports, a route-status report, tool logs and runtime metadata. The agent therefore sees only these structured outcomes, with no shell access and no direct write access to the checkpoint.

Algorithm 1 summarizes the optimization loop. The recipe library P contains the fixed bottleneck-to-action mappings. The evidence record M contains only information collected during the current run. The candidate queue contains mutating ECO and fallback actions. Non-mutating diagnostics update the run state directly and do not enter the acceptance test. Algorithm 1 proceeds in three stages. It first validates the input DCP and returns early if the starting state is invalid [Lines 1–5]. It then commits D_0 as the accepted checkpoint, builds the run state and seeds the candidate queue [Lines 6–8]. Each iteration pops the best action, applies the admissibility filter, and runs an admissible action in isolation from the accepted checkpoint [Lines 9–15]. A validation-clean score gain promotes the candidate; otherwise PACT discards it and logs a rejected case [Lines 16–26]. After every trial it replans from the accepted checkpoint, so failed edits never accumulate, and it returns the best validated checkpoint once the budget or queue is exhausted [Lines 29–32].

E. Validation, Acceptance and Termination

Before scoring, PACT checks that a candidate checkpoint is valid. A valid output DCP is reopenable, fully routed and free of routing errors, and passes the required hold, pulse-width, minimum-period and design-consistency checks.

Among valid candidates, PACT accepts a trial D' only when its setup worst slack improves by at least $\epsilon = 0.5$ ps, $s(D') > s(D_{\text{best}}) + \epsilon$, where $s(\cdot)$ is the worst negative slack (WNS) under the design's original timing constraints, so a temporarily tightened clock never counts as an improvement. This rule keeps the committed trajectory monotonic, while exploration stays diverse: after each trial, PACT updates the timing profile and reranks both local ECO actions and global fallbacks. PACT therefore targets a strong validated result within the budget, not a globally optimal one.

Rejected candidates are discarded, while their actions and validation results are recorded to avoid repeated failures. The run terminates when its time, token or trial budget is exhausted, or when no admissible candidate remains. PACT returns the best validated checkpoint and its reports.

IV. EVALUATION

This section first describes how PACT is implemented around Vivado and RapidWright (Section IV-A) and the benchmark suite (Section IV-B), and then reports the experimental setup, baselines, main results, and an ablation and generalization analysis.

A. Implementation

We implement PACT as an agentic post-route optimization framework built around Vivado and RapidWright. Each supported operation of Section III is realized as a typed skill that renders the action parameters into the underlying Vivado Tcl or RapidWright call, executed in isolation as described in Section III-D. The two backends play complementary roles: Vivado performs checkpoint loading, timing and route report extraction, post-route optimization, checkpoint writing and signoff validation, while RapidWright provides implemented-design inspection and object-level analysis when lower-level checkpoint access is needed.

B. Benchmarks

We collected 35 post-route Vivado design checkpoints targeting the AMD UltraScale+ `xcvu3p-ffvc1517-2-e` device. They come from four sources: public contest checkpoints [26]; open-source RTL projects [27], [30]–[36]; classic academic and HLS-style benchmark designs [28]; and internally generated stress-test checkpoints with deliberately tightened clock targets and near-maximal resource utilization. We designate the †-marked checkpoints in Table III as a held-out test set that PACT optimizes cold, and use the remaining designs for development. PACT's within-run case record does not carry across designs. The split therefore isolates generalization of the action set and recipe planner rather than memorized per-design recipes.

The benchmark suite spans four design groups:

TABLE III
BENCHMARK CHARACTERISTICS, GROUPED INTO THE FOUR CATEGORIES.
TEST DESIGNS ARE MARKED WITH †. $F_{\max}$ IS REPORTED IN MHZ AND
EXTRACTED FROM THE ORIGINAL DCPs.

Benchmark	LUTs	FFs	DSPs	BRAMs	$F_{\max}$
<i>(1) Classic, compact and HLS-style benchmarks</i>					
AMD Mini-ISP [26]	3k	4k	40	12	307.1
LogicNets JSCL [26]	31k	2k	0	0	403.6
Rosetta 3D Rendering [26]	14k	5k	3	0	270.9
Rosetta Digit Recognition [26]	23k	23k	0	161	367.0
Rosetta Optical Flow [26]	34k	37k	42	61	324.9
Rosetta Spam Filter [26]	5k	13k	224	3	437.4
VTR MCML [26]	43k	15k	105	142	62.2
AES [27]	3.4k	3.0k	0	0	334.0
TCAM [28]	156	281	0	0	561.8
VexRiscv Re-place [29]	2k	1k	4	6	310.2
OpenRISC 1200† [30]	4.6k	2.5k	4	4	209.0
<i>(2) Compute-intensive datapath designs</i>					
FINN RadioML [26]	74k	46k	0	25	284.9
FSA Systolic Array [31]	373.8k	29.2k	256	0	82.8
Double-Clock FFT-L [32]	10.8k	26.2k	0	32	442.5
Double-Clock FFT-M [32]	5.4k	13.1k	0	16	456.4
Double-Clock FFT-S [32]	1.4k	3.3k	0	4	477.1
RaygenTop [28]	986	1.7k	9	0	451.3
Verilog CORDIC-L [33]	23.4k	23.9k	0	0	456.6
Verilog CORDIC-M [33]	11.7k	12.0k	0	0	473.7
Verilog CORDIC-S [33]	750	823	0	0	682.1
NVDLA Large† [34]	241.5k	30.6k	0	0	158.1
NVDLA XL [34]	272.6k	33.7k	0	0	152.5
NVDLA Basic [34]	30.2k	722	0	0	189.9
<i>(3) Large system-level designs</i>					
BOOM SoC [26]	227k	98k	61	161	48.2
CoreScore-500 [26]	100k	120k	0	250	344.2
ISPD16 Example2 [26]	289k	234k	200	384	107.6
AXIS Fabric† [35]	76.5k	68.2k	0	144	179.1
AXIS Fabric-Hard† [35]	229.8k	204.6k	0	432	140.0
Vortex Xbar Large† [36]	215.5k	156.9k	0	0	146.8
Vortex Xbar Basic [36]	41.4k	27.3k	0	0	210.5
<i>(4) Generated backend-focused checkpoints</i>					
Congestion XL†	84.6k	104.1k	48	64	305.2
FFT-CORDIC-L†	60.7k	77.4k	32	48	338.4
FPL26 Self-built	10.7k	7.9k	144	24	235.0
FFT-CORDIC-M	30.9k	39.1k	16	24	343.9
SoC-L†	43.7k	52.8k	32	32	326.9

- classic, compact and HLS-style benchmarks;
- compute-intensive datapaths, including FFT, CORDIC, systolic-array and rendering designs;
- large system-level designs such as SoCs, processors, accelerators and network fabrics, with global placement effects and high-fanout routing;
- generated backend-focused checkpoints that stress congestion and mixed-resource paths.

Together, these groups test generalization across multiple post-route timing-failure modes rather than a single narrow pattern.

C. Experimental Setup

We evaluate all methods on post-route Vivado checkpoints targeting the `xcvu3p-ffvc1517-2-e` device, using Vivado 2025.1 and RapidWright 2025.1 as the backend toolchain and GPT-5.5 as the underlying model for the agentic methods. For fair comparison, all runs share the same resource limit—eight cores and 32 GB RAM on an Ubuntu 22.04 machine with an Intel Core i7-12700 processor—and the same validation policy C . The primary metric is the best validation-clean $F_{\max}$

obtained within the tuning budget, where a result is *validation-clean* only when it passes the full policy of Section III-E: a reopenable, fully routed DCP with zero routing errors; non-negative hold, pulse-width and min-period slack and no failing endpoints; a setup/ $F_{\max}$ measured against the original target clock, with $F_{\max} = 1000/(T - \text{WNS})$ MHz for target-clock period T ; and preserved primary I/O and clock definitions (functional equivalence holds by construction).

The two agentic methods, PACT and the Codex Agent, run under a one-hour wall-clock budget per design. A PACT run rarely uses the full budget. Its loop exits once no admissible candidate remains: queued actions are exhausted, infeasible within the budget, or suppressed by the case record. PACT then returns the best validated checkpoint, so most runs finish well before the cap (Section IV-E2). DATuner is not subject to this cap and instead terminates on its own search budget (Section IV-D).

Across all experiments, PACT draws only from the bounded backend-action set of Table II and commits only route-legal, validation-clean improvements. All reported $F_{\max}$ values use the original clock and timing-exception constraints, without constraint relaxation.

D. Baselines

DATuner. We use DATuner as a search-based autotuning baseline, representing the class of FPGA tool-flow tuners that explore a parameterized implementation space using search. We run its bandit-based space-partitioning algorithm over a 47,250-point implementation space: Vivado `opt`, `place`, `phys_opt` and `route` directives, a fanout limit and a clock-tightening target factor. We ran `place-and-route` for each trial on the same benchmark designs and evaluated it under the same validation gates as PACT, but without the one-hour wall-clock budget imposed on the agentic methods. This favorable setting tests whether PACT can reach competitive or better validation-clean results more time-efficiently through structured post-route checkpoint actions.

Codex Agent. We use Codex command-line interface (CLI) 0.136.0 as a knowledge-and-tool-matched LLM-agent baseline. The baseline receives the same optimization knowledge, Vivado and RapidWright Model Context Protocol (MCP) servers, reports, and checkpoint inputs as PACT, and runs GPT-5.5 under the same one-hour budget. However, it does not use PACT’s constrained execution harness, structured action representation, validation-gated acceptance or rollback discipline. This comparison isolates the contribution of PACT’s harness from the contribution of LLM reasoning, domain knowledge and tool availability.

E. Main Results

1) Frequency Improvement: Figure 2 compares the best validation-clean $F_{\max}$ obtained by each method, normalized to the original post-route checkpoint. PACT achieves the strongest overall result, improving $F_{\max}$ by a geometric mean of +22.30% across the 35-design benchmark suite. In comparison, DATuner improves by +15.14% and the Codex

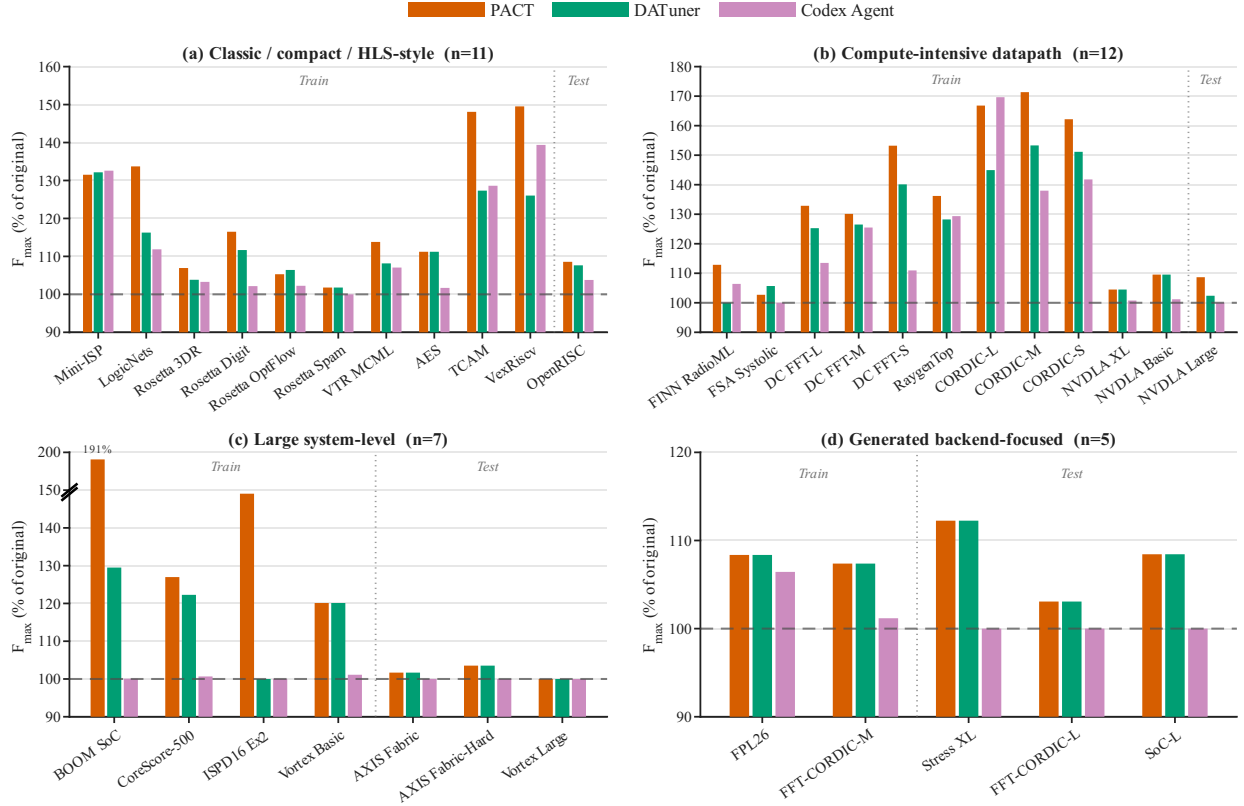


Fig. 2. Per-design $F_{\max}$ achieved by PACT, DATuner and the Codex Agent, normalized to the original post-route checkpoint (Original = 100%, dashed). The dotted divider separates the development designs and held-out test designs.

Agent by +9.78%. Per design, PACT obtains the best or tied-best result on 27 of 35 benchmarks, showing that its aggregate gain is not driven by a small number of outliers.

PACT also reaches these results with limited backend exploration. On average, PACT evaluates only 2.6 checkpoint-modifying backend actions per DCP, excluding read-only diagnostic probes. In comparison, DATuner explores about 12 full place-and-route trials per design, while the free-form Codex Agent issues roughly 60 tool calls per design. Although these counts reflect different execution granularities, they show that PACT concentrates the search into a small number of targeted, validation-gated checkpoint edits.

The largest improvements appear on checkpoints with recoverable timing margin. For example, PACT improves BOOM SoC from 48.2 to 91.9 MHz, raises the Verilog CORDIC family by up to 1.71 $\times$ and improves TCAM from 561.8 to 831.9 MHz. On congestion-bound designs, PACT often preserves the original implementation rather than accepting unsafe or marginal edits. Vortex Xbar Large shows this behavior, as all methods find limited additional margin.

2) *Time Efficiency*: PACT achieves its frequency gains within the one-hour wall-clock budget on all 35 designs, with a mean runtime of 36 minutes (2140 s). In contrast, DATuner is run without this cap, giving the search-based autotuner a favorable setting, yet still requires much longer runtimes: 4.5 hours on average (16070 s) and up to 12.5 hours (45162 s). As

shown in Figure 3(a), DATuner lies substantially to the right of PACT on the runtime axis, running slower on 30 of the 35 shared designs by a geometric mean of 6.4 $\times$.

The additional runtime does not translate into better quality: on the 35 designs completed by both methods, PACT achieves higher $F_{\max}$ on 26 designs and DATuner on seven, so PACT delivers comparable-or-better timing quality with bounded runtime and substantially lower wall-clock cost. The Codex Agent also fits within the one-hour budget with a lower mean runtime of 25 minutes (1524 s) but, as noted above, achieves lower frequency gains, leaving PACT with the best time-quality tradeoff among the evaluated methods.

3) *Token Efficiency*: The Codex Agent’s free-form harness costs **\$3.81 per DCP on average**, whereas PACT costs only **\$0.16 per DCP on average** on the same 35 designs. On a per-DCP basis this is a 24.5 $\times$ cost reduction for PACT, under the same GPT-5.5 model, one-hour wall-clock budget, optimization knowledge, checkpoint inputs and Vivado/RapidWright tool access—the main difference being the execution harness.

As shown in Figure 3(b), PACT is consistently cheaper while also achieving higher $F_{\max}$ on 32 of the 35 shared designs. This efficiency comes from PACT’s constrained backend harness. PACT uses structured actions, validation-gated acceptance, rollback and a within-run case record. These mechanisms avoid repeated unsafe or ineffective tool exploration. By contrast, the free-form Codex Agent repeatedly re-

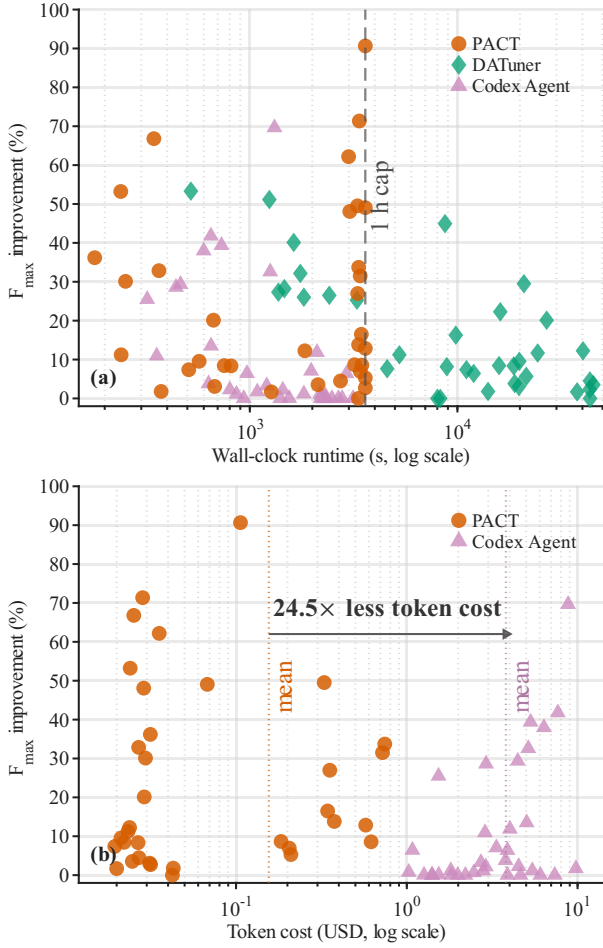


Fig. 3. (a) Quality–runtime and (b) quality–cost tradeoffs: per-design $F_{\max}$ improvement over the original (%) versus wall-clock runtime and token cost for PACT, DATuner and the Codex Agent. Dashed line: one-hour cap; dotted lines: each method’s average per-design cost.

TABLE IV
GEOMETRIC-MEAN $F_{\max}$ IMPROVEMENT (%) ON THE 27 DEVELOPMENT AND 8 HELD-OUT TEST DESIGNS. *Scripted* REPLACES THE LLM WITH PACT’S RULE LAYER OVER THE SAME ACTIONS, GATES AND BUDGET; *phys_opt* IS A PURE VIVADO DIRECTIVE SWEEP WITH NO AGENT.

Method	Dev (27)	Test (8)	All (35)
PACT (full)	+27.70	+5.70	+22.30
Scripted (no LLM)	+16.78	+5.54	+14.11
phys_opt sweep	+3.53	+1.12	+2.98
DATuner	+18.39	+4.80	+15.14
Codex Agent	+12.70	+0.47	+9.78

derives context and re-validates trial-and-error actions. This raises token consumption without matching PACT’s timing gains. DATuner has no token cost (it is not LLM-based) but pays for search through substantially longer wall-clock runtime (Figure 3(a)).

F. Ablation and Generalization Analysis

We split the 35 designs into 27 development and 8 held-out test designs that the agent optimizes cold, and add two

ablations (Table IV). A pure *phys_opt_design* directive sweep, Vivado’s own post-route optimizer, with no agent or RapidWright, reaches only +2.98%, far below PACT, so the commercial backend alone cannot express the local edits PACT applies. Replacing the LLM with PACT’s deterministic rule layer (*Scripted*) gives +14.11%: agentic action selection supplies the remaining gap, mostly on development designs. On the held-out designs PACT (+5.70%) and *Scripted* (+5.54%) are comparable and both exceed the uncapped DATuner (+4.80%), so the structured action set and validation harness, rather than the LLM, carry generalization to unseen checkpoints.

We further run a leave-one-out test to measure cross-design evidence transfer. The test uses five designs from three families: Verilog CORDIC-L/M, Double-Clock FFT-L/M, and the held-out NVDLA Large. For each target design, we compare three settings: no prior evidence, evidence from family members, and evidence from the target design itself. The final $F_{\max}$ values show little variation across the three settings; for example, CORDIC-L reaches (+26.7%) in each setting.

These similar outcomes suggest that the transferable cases add little decision value beyond the recipe planner. For these designs the useful cross-design knowledge is compact: once the profile exposes the dominant bottleneck, the curated recipe and the retrieved cases favor the same action. The case record then mainly prunes repeated or invalid trials within a run and keeps the trajectory auditable. Larger cross-design gains likely require finer EDA-report semantics, for example when several RapidWright ECOs fit one profile; such cases are rare in this suite. Overall, PACT’s gains come mainly from the recipe planner, the structured action set and validation-gated execution.

V. CONCLUSION

This paper presented PACT, a post-route agentic checkpoint-tuning framework that operates on implemented Vivado checkpoints through structured Vivado and RapidWright actions. Each candidate edit is a validation-gated transition. PACT commits it only when it improves timing while preserving routing legality, hold and pulse-width safety and checkpoint usability, and rolls it back otherwise; functional equivalence holds by construction. Across 35 UltraScale+ post-route checkpoints PACT improves validation-clean $F_{\max}$ by a geometric mean of +22.30%, ahead of DATuner (+15.14%) and the Codex Agent (+9.78%), while running 6.4× faster than DATuner and at 24.5× lower token cost than the Codex Agent.

The results suggest that post-route timing closure is limited not only by search time but also by action granularity and validation discipline. PACT’s gains come from local checkpoint edits and a recipe planner beyond the reach of global search. Future work will extend PACT to cross-design evidence transfer, multi-clock, cross-die and power-aware optimization.

REFERENCES

- [1] AMD, *Vivado Design Suite User Guide: Implementation (UG904)*, Advanced Micro Devices, Inc., 2025, document ID: UG904, accessed 2026-05-30, <https://docs.amd.com/r/en-US/ug904-vivado-implementation>.
- [2] AMD, *Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906)*, Advanced Micro Devices, Inc., 2025, document ID: UG906, accessed 2026-05-30, <https://docs.amd.com/r/en-US/ug906-vivado-design-analysis>.
- [3] AMD, *UltraFast Design Methodology Guide for FPGAs and SoCs (UG949)*, Advanced Micro Devices, Inc., 2025, document ID: UG949, accessed 2026-05-30, <https://docs.amd.com/r/en-US/ug949-vivado-design-methodology>.
- [4] W. Jiang, V. A. Chhabria and S. S. Sapatnekar, “IR-aware ECO timing optimization using reinforcement learning,” *arXiv:2402.07781*, 2024.
- [5] A. Azadi, P. Rigge, E. Mahintorabi and K. B. Kent, “Graph-based ECO and patch generation for high-level synthesis,” *arXiv:2604.14248*, 2026.
- [6] RapidWright Project, *Inserting and Routing a Debug Core As An ECO*, 2025, accessed 2026-06-18, https://www.rapidwright.io/docs/ECO_Insert_Route_Debug.html.
- [7] RapidWright Project, *RapidWright Documentation*, 2025, accessed 2026-05-30, <https://www.rapidwright.io/docs/RapidWright.pdf>.
- [8] Y. Zhou, P. Maidee, C. Lavin, A. Kaviani and D. Stroobandt, “RWRRoute: An open-source timing-driven router for commercial FPGAs,” *ACM Trans. Reconfigurable Technol. Syst.* 15(1) (2022), pp. 1-27.
- [9] AMD, *Vivado Design Suite User Guide: Using Tcl Scripting (UG894)*, Advanced Micro Devices, Inc., 2025, document ID: UG894, accessed 2026-05-30, <https://docs.amd.com/r/en-US/ug894-vivado-tcl-scripting>.
- [10] AMD, *Vivado Design Suite Tutorial: Design Analysis and Closure Techniques (UG938)*, Advanced Micro Devices, Inc., 2025, document ID: UG938, accessed 2026-05-30, <https://docs.amd.com/r/en-US/ug938-vivado-design-analysis-closure-tutorial>.
- [11] N. Kapre, H. Ng, K. Teo and J. Naude, “InTime: A machine learning approach for efficient selection of FPGA CAD tool parameters,” *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays*, 2015, pp. 23-26.
- [12] C. Xu, G. Liu, R. Zhao, S. Yang, G. Luo and Z. Zhang, “A parallel bandit-based approach for autotuning FPGA compilation,” *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays*, 2017, pp. 157-166.
- [13] E. Ustun, S. Xiang, J. Gui, C. Yu and Z. Zhang, “LAMDA: Learning-assisted multi-stage autotuning for FPGA design closure,” *Proc. IEEE Int. Symp. Field-Programmable Custom Computing Machines*, 2019, pp. 74-77.
- [14] J. Pan, G. Zhou, C.-C. Chang, I. Jacobson, J. Hu and Y. Chen, “A survey of research in large language models for electronic design automation,” *ACM Trans. Design Autom. Electron. Syst.* 30(3) (2025), pp. 1-21.
- [15] M. Liu, T.-D. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben *et al.*, “ChipNeMo: Domain-adapted LLMs for chip design,” *arXiv:2311.00176*, 2023.
- [16] A. Ghose, A. B. Kahng, S. Kundu and Z. Wang, “ORFS-agent: Tool-using agents for chip design optimization,” *Proc. ACM/IEEE Int. Symp. Machine Learning for CAD*, 2025, pp. 1-13.
- [17] Y. Ouyang, Y. Luo, D. Zuo and Y. Ma, “Retrieve, schedule, reflect: LLM agents for chip QoR optimization,” *arXiv:2603.13767*, 2026.
- [18] J. Nainani, C.-T. Ho, A. Dhurka and H. Ren, “Timing analysis agent: Autonomous multi-corner multi-mode timing debugging with timing debug relation graph,” *arXiv:2504.11502*, 2025.
- [19] M. A. Elgammal, K. E. Murray and V. Betz, “RLPlace: Using reinforcement learning and smart perturbations to optimize FPGA placement,” *IEEE Trans. Computer-Aided Design Integr. Circuits Syst.* 41(8) (2022), pp. 2532-2545.
- [20] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg and R. Karri, “AutoChip: Automating HDL generation using LLM feedback,” *arXiv:2311.04887*, 2023.
- [21] C.-T. Ho, H. Ren and B. Khailany, “VerilogCoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree-based waveform tracing tool,” *Proc. AAAI Conf. Artificial Intelligence*, 2025, pp. 300-307.
- [22] N. Mashnoor, M. Akyash, H. Kamali and K. Azar, “TimelyHLS: LLM-based timing-aware and architecture-specific FPGA HLS optimization,” *Proc. IEEE Int. Conf. Omni-layer Intelligent Systems*, 2025, pp. 1-6.
- [23] M. Moraru, K. Kamalakkannan, J. Dominguez-Trujillo, P. Diehl, A. Barai, J. Loiseau, Z. K. Baker, H. Pritchard and G. M. Shipman, “LAAF: LLM-based agents for accelerated FPGA design,” *arXiv:2602.06085*, 2026.
- [24] P. Esmaili, T. Martin, S. Areibi and G. Grewal, “Guiding FPGA detailed placement via reinforcement learning,” *Proc. IFIP/IEEE Int. Conf. Very Large Scale Integration*, 2022, pp. 1-6.
- [25] U. Farooq, N. Ul Hasan, I. Baig and M. Zghaibeh, “Efficient FPGA routing using reinforcement learning,” *Proc. Int. Conf. Information and Communication Systems*, 2021, pp. 106-111.
- [26] Xilinx, “Agentic FPGA Backend Optimization Competition @ FPL’26,” 2026, accessed 2026-05-30, https://xilinx.github.io/fpl26_optimization_contest/.
- [27] secworks, “AES Verilog core,” accessed 2026-06-02, <https://github.com/secworks/aes>.
- [28] Verilog-to-Routing Project, “VTR benchmarks,” accessed 2026-06-02, <https://docs.verilogtorouting.org/en/latest/vtr/benchmarks/>.
- [29] SpinalHDL, “VexRiscv: A FPGA-friendly 32-bit RISC-V CPU implementation,” accessed 2026-06-02, <https://github.com/SpinalHDL/VexRiscv>.
- [30] OpenRISC, “OR1200: OpenRISC 1200 implementation,” accessed 2026-06-02, <https://github.com/openrisc/or1200>.
- [31] VCA-EPFL, “FSA: Fusing FlashAttention within a single systolic array,” accessed 2026-06-02, <https://github.com/VCA-EPFL/FSA>.
- [32] D. Gisselquist, “A generic pipelined FFT core generator,” accessed 2026-06-02, <https://github.com/ZipCPU/dblckfft>.
- [33] OpenCores, “Configurable CORDIC core in Verilog,” accessed 2026-06-02, https://opencores.org/projects/verilog_cordic_core.
- [34] NVIDIA, “NVDLA open source hardware,” accessed 2026-06-02, <https://github.com/nvdla/hw>.
- [35] A. Forencich, “Corundum: Open-source FPGA-based NIC and platform for in-network compute,” accessed 2026-06-02, <https://github.com/corundum/corundum>.
- [36] Vortex GPGPU, “Vortex: Open-source RISC-V GPGPU,” accessed 2026-06-02, <https://github.com/vortexgpgpu/vortex>.
- [37] Anonymous repository, <https://anonymous.4open.science/r/PACT-552C>.